

Project Proposal

Title: Personal Finance Tracker Using Microsoft Azure

Course: CS 4037 – Introduction to Cloud Computing

Instructor: Muhammad Sudais

Group Members:

- Mustafa
- Laiba
- Hafsah
- Umair
- Shahzaib
- Naveed
- Faiq

1. Project Overview

The Personal Finance Tracker is a cloud-based application that allows users to securely manage their expenses, track income, set budgets, and visualize spending patterns in real-time. The solution leverages Microsoft Azure services for scalability, availability, and security. Users will have role-based access (standard user vs. admin), and the application will provide insights into financial health through dashboards and reports.

The application will consist of:

- Frontend (Web App): User-friendly dashboard for adding/viewing expenses, income, and budgets.
- Backend (Azure Functions & APIs): Business logic for transaction processing, categorization, and analytics.
- Database (Cosmos DB): To store user accounts, transactions, and budget history.
- Storage (Blob): For storing receipts, invoices, and financial documents uploaded by users.

2. Objectives

- To design and implement a cloud-native application using Microsoft Azure.
- To provide secure user authentication and role-based access control with Azure Active Directory.
- To integrate Azure Functions for categorizing transactions and processing uploaded receipts.

- To demonstrate monitoring, logging, and performance optimization using Azure Monitor and Application Insights.
- To ensure scalability, resilience, and cost optimization of the solution.

3. Project Requirements

3.1 Functional Requirements

- User Authentication: Register/login via Azure AD, role-based access (user/admin).
- Expense & Income Management: Add, update, delete, and view transactions; categorize expenses.
- Budgeting: Set monthly budgets and receive notifications if nearing/exceeding budget.
- Receipt Uploads: Upload and store receipts/documents in Azure Blob Storage.
- API Integration: Expose APIs via Azure API Management, Azure Function to process receipts.
- Monitoring & Logging: Azure Monitor and Application Insights for performance and error tracking.
- Security: Manage secrets via Azure Key Vault.

3.2 Non-Functional Requirements

- Performance: Handle 100+ concurrent users.
- Scalability: Auto-scale using Azure App Service.
- Availability: Target uptime of 99.95%.
- Security: End-to-end encryption (HTTPS), Azure AD, Key Vault integration.
- Cost Optimization: Use consumption-based services to minimize costs.

4. Azure Services to be Used

- Compute: Azure App Service, Azure Functions.
- Database: Azure Cosmos DB.
- Storage: Azure Blob Storage.
- Security: Azure AD, Azure Key Vault.
- API Management: Secure API exposure.

- Monitoring & Diagnostics: Azure Monitor, Application Insights.
- DevOps (Optional): Azure DevOps for CI/CD.

5. Project Milestones

- Milestone 1 (Week 2): Submit proposal with group details, app idea, and Azure services.
- Milestone 2 (Week 4): Submit application design + architecture diagram.
- Milestone 3 (Week 8): Deploy initial version with authentication + database.
- Milestone 4 (Week 12): Implement full features (budgeting, receipts, APIs, monitoring).
- Milestone 5 (Week 16): Final submission + project presentation.

6. Deliverables

- Proposal document (this file).
- Application source code (hosted on GitHub/Azure DevOps).
- Azure deployment details and cost estimates.
- Final report (architecture, challenges, and learnings).
- Final presentation (10 minutes demo + architecture walkthrough).

7. Architecture Diagram (Draft)

[User] → [Azure App Service (Frontend)] → [Azure API Management] → [Azure Functions]
→ [Azure Cosmos DB]
→ [Azure Blob Storage]
→ [Azure Key Vault]
→ [Azure Monitor & Application Insights]

8. Expected Outcomes

- A secure, scalable, and cost-optimized personal finance application.
- Hands-on experience with multiple Azure services.
- Understanding of best practices in cloud architecture and DevOps.